ElasticSearch

安装使用

- 因安全原因，ES不允许root用户运行，添加新用户
  - usersadd XXX
  - passwd XXX
  - 开为其添加权限
    chown -R user:group file
- 修改配置文件elasticsearch.yml
  - cluster.name: my-application
    node.name: 1
    network.host: 0.0.0.0
    http.port: 9217
    cluster.initial_master_nodes: ["node-1"]
  - cluster.name：配置elasticsearch的集群名称，默认是elasticsearch。建议修改成一个有意义的名称
  - node.name：节点名，elasticsearch会默认随机指定一个名字，建议指定一个有意义的名称，方便管理
  - network.host：设置为0.0.0.0允许外网访问
  - http.port：Elasticsearch提供http访问端口
  - cluster.initial_master_nodes：初始化新的集群时需要此配置来选举master
- 切换到root用户修改最大可创建文件数
  - vim /etc/security/limits.conf
    - user soft nofile 65536
    - user hard nofile 65536
  - vim /etc/security/limits.d/20-nproc.conf
    - user soft nofile 65536
    - user hard nofile 65536
    - * hard nproc 4096
  - 虚拟内存 vim /etc/sysctl.conf
    - vm.max_map_count=655360
    - sysctl -p 重新加载系统配置
- 以上设置完毕，启动ES即可使用
- Kibana插件、head插件(node、grunt、head等)的安装不赘述

核心概念

- 索引（index）
  - ElasticSearch存储数据的地方，可以理解成关系型数据库中的数据库
- 映射（mapping）
  - mapping定义了每个字段的类型、字段所使用的分词器等，相当于关系型数据库中的表结构。
- 文档（document）
  - Elasticsearch中的最小数据单元。常以Json格式显示。一个document相当于关系型数据库中的一行数据。
- 倒排索引
  - 一个倒排索引由文档中所有不重复词的列表构成，对于其中每个词，对应一个包含它的文档id列表。
- 类型（type）
  - 一种type就像一类表，如用户表、角色表等。在Elasticsearch7版本以后type为_doc。

脚本操作

- 操作索引
  - PUT
    - http://ip:端口/索引名称
  - 查询
    - GET http://ip:端口/索引名称 # 查询单个索引信息
    - GET http://ip:端口/索引名称1,索引名称2... # 查询多个索引信息
    - GET http://ip:端口/_all # 查询所有索引信息
  - 删除索引
    - DELETE http://ip:端口/索引名称
  - 关闭、打开索引
    - POST http://ip:端口/索引名称/_close
    - POST http://ip:端口/索引名称/_open
- 操作映射
  - 简单数据类型
    - 字符串
      - text：会分词，不支持聚合
      - keyword：不会分词，将全部内容作为一个词条，支持聚合
    - 数值
      - long：带符号的64位整数
      - integer：带符号的32位整数
      - short：带符号的16位整数
      - byte：带符号的8位整数
      - double：双精度64位小数
      - float：单精度32位小数
      - half_float：半精度16位小数
      - scaled_float：由a支持的有限浮点数long，由固定double此刚的不稳定
    - 布尔：boolean
    - 二进制：binary
    - 范围类型
      - integer_range
      - float_range
      - long_range
      - double_range
      - date_range
    - 日期：date
  - 复杂数据类型
    - 数组：[ ]
    - 对象：{ }
  - 添加映射示例
    - PUT /person/_mapping
      {
      "properties": {
      "name": {
      "type": "text"
      },
      "age": {
      "type": "integer"
      }
      }
      }
  - 创建索引并添加映射示例
    - PUT /person1
      {
      "mappings": {
      "properties": {
      "name": {
      "type": "text"
      },
      "age": {
      "type": "integer"
      }
      }
      }
      }
  - 添加字段示例
    - PUT /person1/_mapping
      {
      "properties": {
      "name": {
      "type": "text"
      },
      "age": {
      "type": "integer"
      }
      }
      }
- 操作文档
  - 添加文档，指定id
    - POST /person1/_doc/2
      {
      "name":"张三",
      "age":18,
      "address":"北京"
      }
  - 添加文档，不指定id
    - POST /person1/_doc/
      {
      "name":"张三",
      "age":18,
      "address":"北京"
      }
  - 查询所有的文档
    - GET /person1/_search
  - 删除指定id文档
    - DELETE /person1/_doc/1

分词器

- IKAnalyzer
  - 安装
  - 使用
    - ik_max_word 细粒度
      - GET /_analyze
        {
        "analyzer": "ik_max_word",
        "text": "乒乓球明年总冠军"
        }
    - ik_smart 粗粒度
      - GET /_analyze
        {
        "analyzer": "ik_smart",
        "text": "乒乓球明年总冠军"
        }
    - 完整流程示例
      - 创建索引，添加映射，并指定分词器为分词器
        - PUT person2
          {
          "mappings": {
          "properties": {
          "name": {
          "type": "keyword"
          },
          "address": {
          "type": "text",
          "analyzer": "ik_max_word"
          }
          }
          }
          }
      - 添加文档
        - POST /person2/_doc/1
          {
          "name":"张三",
          "age":18,
          "address":"北京海淀区"
          }
        - POST /person2/_doc/2
          {
          "name":"李四",
          "age":18,
          "address":"北京朝阳区"
          }
        - POST /person2/_doc/3
          {
          "name":"王五",
          "age":18,
          "address":"北京昌平区"
          }
      - 查询映射
        - GET person2
      - 查看分词效果
        - GET /_analyze
          {
          "analyzer": "ik_max_word",
          "text": "北京海淀区"
          }
      - 词条查询：term
        查询person2中匹配到"北京"两字的词条
        - GET person2/_search
          {
          "query": {
          "term": {
          "address": {
          "value": "北京"
          }
          }
          }
          }
      - 全文查询：match
        - GET /person2/_search
          {
          "query": {
          "match": {
          "address": "北京昌平"
          }
          }
          }

SpringBoot整合ES

- 使用流程
  - 导包
  - 创建配置类
    - @Configuration
      @ConfigurationProperties(prefix=" elasticsearch")
      public class ElasticSearchConfig {
      private String host;
      private int port;
      @Bean
      public RestHighLevelClient client(){
      return new RestHighLevelClient(
      RestClient.builder(
      new HttpHost(host,port,"http")
)
      );
      }
      }
  - 配置文件编写
  - 注入使用
    - @Autowired
      RestHighLevelClient client;
- 索引操作
  - 创建索引
    - IndicesClient indices = client.indices();
      CreateIndexRequest getRequest=new CreateIndexRequest("索引名");
      CreateIndexResponse response = indices.get(getRequest, RequestOptions.DEFAULT);
      System.out.println(response.getAcknowledged());
  - 查询索引
    - IndicesClient indices = client.indices();
      GetIndexRequest getIndexRequest=new GetIndexRequest("索引名");
      GetIndexResponse response = indices.get(getRequest, RequestOptions.DEFAULT);
      Map<String, MappingMetaData> mappings = response.getMappings();
      //for 遍历foreach
      for (String key : mappings.keySet()) {
      System.out.println(key+"==>"+ mappings.get(key).getSourceAsMap());
      }
  - 删除索引
    - IndicesClient indices = client.indices();
      DeleteIndexRequest deleteRequest=new DeleteIndexRequest("theima");
      AcknowledgedResponse delete = indices.delete(deleteRequest, RequestOptions.DEFAULT);
      System.out.println(delete.isAcknowledged());
  - 查引是否存在
    - IndicesClient indices = client.indices();
      GetIndexRequest getIndexRequest=new GetIndexRequest("theima");
      boolean exists = indices.exists(getIndexRequest, RequestOptions.DEFAULT);
      System.out.println(exists);
- 文档操作
  - 添加文档
    - 添加文档,使用map作为数据
      - Map<String, Object> map=new HashMap<>();
        map.put("name","张三");
        map.put("age",18");
        map.put("address","北京二环");
        IndexRequest request=new IndexRequest("person1","1").source(map);
        IndexResponse response = client.index(request, RequestOptions.DEFAULT);
        System.out.println(response.getId());
    - 添加文档,使用对象作为数据
      - Person person=new Person();
        person.setId("2");
        person.setName("李四");
        person.setAge(20);
        person.setAddress("北京三环");
        String data = JSON.toJSONString(person);
        IndexRequest request=new IndexRequest("person1","2").source(data, XContentType.JSON);
        IndexResponse response = client.index(request, RequestOptions.DEFAULT);
        System.out.println(response.getId());
  - 修改文档
    - 添加文档时，如果id存在则修改，id不存在则添加
    - GetRequest indexRequest=new GetRequest("person1","2");
      GetResponse response = client.get(indexRequest, RequestOptions.DEFAULT);
      System.out.println(response.getSourceAsString());
  - 查询文档
  - 删除文档
    - DeleteRequest deleteRequest=new DeleteRequest("person1","1");
      DeleteResponse response = client.delete(deleteRequest, RequestOptions.DEFAULT);
      System.out.println(response.getId());